

The Governed Factory

One artifact plane, five gates, and two loops that learn – the design that implements all ten S7 feature priorities as genuinely running, deterministic mechanisms. No black box: every artifact is hashed, chained, traceable, and gated before it moves.



DESIGN – approved 2026-08-06 · not yet built **PLANNED**

This page describes the approved design, not a running system. Anything shown here ships only when its build step lands; the sibling architecture pages describe what runs today. Full spec: [docs/superpowers/specs/2026-08-06-governed-factory-design.md](#).

00 Four decisions that shape everything

Real, deterministic – not staged

The self-healing loop and the accuracy loop run for real on real run data. No LLM required; nothing in them is badged STAGED.

Three epics show the trend

EPIC-S7-001 plus two new synthetic epics (sponsor billing statements, member document e-consent), each with committed “actuals”. Error narrows epic over epic.

Minimal real downstream lane

A small synthetic target app in the repo. Patches apply for real, pytest runs red→green for real, release is a gated ledger event. Patch content pre-authored and labelled.

Artifact plane is the spine

All ten features are views or checks over one shared file plane – not ten separate subsystems.

01 The artifact plane

Every stage writes a JSON envelope to a deterministic path. The header carries a SHA-256 content hash and `derived_from` pointers at the upstream artifacts it was built from – which turns the plane into a provenance chain, makes staleness a hash comparison, and makes interrupted runs resumable.

```
artifacts/                                ← gitignored, regenerated by running
  EPIC-S7-002/
    assess/assessment.json
    design/dfd.json er.json
    stories/US-021.json ...
    verify/verdict.json ← Gate 3 output
    build/ test/ docs/ release/
    calibration/record.json
  ledger.jsonl ← append-only, hash-chained

committed: crs/ (epics + actuals),
rules/, skills/, target_app/, patches/
```

ledger.jsonl – the provenance ledger

One line per artifact event; each line carries the SHA-256 of the previous line. Editing history breaks the chain visibly – tamper-evident with zero infrastructure. A broken chain raises at load, naming the offending line.

meta.derived_from – the traceability matrix

Walk the pointers epic → design → story → task → test → release, in either direction. A defect traces backward in one lookup.

Staleness detection

Upstream hash no longer matches what downstream recorded? Downstream is stale, flagged in ledger and UI – and Gate 4 refuses release until it is regenerated. Regeneration is explicit (`--refresh`); it marks things stale rather than deleting them.

02 The four layers, as directories

LAYER	WHERE	WHAT
Rules	s7_delivery/rules/delivery-rules.md	Loaded identically into every stage's prompt prefix – the existing <code>common/prompt.py</code> ordering.
Skills	s7_delivery/skills/<stage>.md + .history.jsonl	Per-stage instruction files, versioned. The thing self-healing amends.
Workflows	s7_delivery/pipeline.py	Stage orchestration and gate enforcement – gates live here so no surface can route around them.
Orchestrator	CLI runner · console	Thin views over the same orchestration. Neither holds logic.

03 Five gates, one pipeline

Nothing advances without passing a gate, and no phase approves its own work. Gates fail loudly with named reasons – and those reasons are what the healing loop consumes.

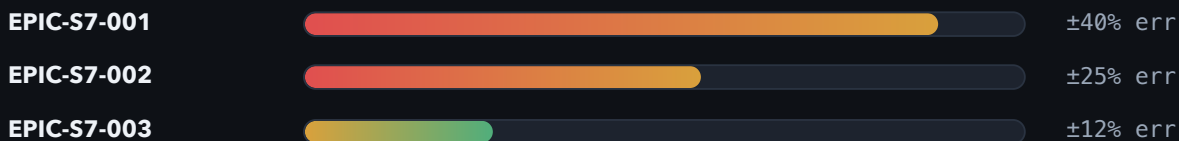


Honesty rule on G3

No LLM access yet, so Gate 3 is a **rule-based verifier and is labelled exactly that** in every surface – “rule-based checks today; second-model review when LLM access lands.” It is never presented as AI. The module boundary is where a second model drops in later, and only that module changes.

04 The accuracy loop – each epic makes the next one better

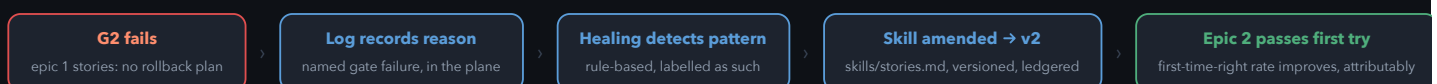
On epic completion, a calibration artifact records predicted vs. actual per stream and the run's error. Per-stream correction factors update (exponentially weighted), and the next epic's assessment applies them – recording **both raw and corrected estimates**, so the adjustment is visible, never hidden.



Illustrative targets – the demo shows whatever the real computation produces over the committed actuals. This is the concrete form of the agreed answer on estimates (“grounded in historical delivery data”) and feeds the estimation-accuracy KPI directly.

05 The self-healing loop – the factory fixes its own skills

The activity log records every stage outcome, including gate failures with their reasons. After each run, healing scans for recurring failure patterns and appends a versioned amendment to the responsible skill file – recorded in the ledger with the triggering log entries as `derived_from`.



The chain *log entry* → *amendment* → *improved outcome* is walkable in the ledger. This is change management running for real: impact recorded, amendment versioned, stability verified by the next run.

06 Contract, surfaces, tests

Contract – all additive

UserStory += purpose, impacts,
feature_flag, rollback_plan · Task
+= task_type, patch_path · new:
ArtifactMeta, Verdict,
CalibrationRecord,
SkillAmendment,
DownstreamResult. Existing 60 tests
stay green.

Surfaces

CLI: python -m
s7_delivery.run <EPIC-ID> –
full run, printed transcript, pytest-
assertable. Console: three new
panels – Ledger/Traceability, KPIs
(accuracy + first-time-right), Factory
(skills + amendment history). Both
human gates stay.

Tests – one file per mechanism

Five gates block/unblock ·
envelope hashing · ledger tamper
detection · staleness propagation ·
calibration narrows (asserted) ·
healing amendment lands and next
run passes · downstream
red→green. All offline, no API key.

07 Build order

Respecting the repo's hard sequencing rules: contract before downstream; anything the lane carries lands in the contract first.

1 Artifact plane + ledger
envelopes, hashing, chain, staleness

2 Contract + G0/G2
story-quality fields and completeness checks

3 G3 verifier
verdict artifacts; downstream refuses unverified

4 Target app + lane + G4
build → test → docs → gated release

5 Calibration + two epics
EPIC-S7-002 / 003 with actuals

6 Healing
log patterns → versioned skill amendments

7 Console panels
ledger, KPIs, factory

8 Deliverables v2
deck, teleprompt, HTML pages – describing what
then runs

08 What this delivers, feature by feature

- ✓ Gated pipeline (5 gates) – G0-G4, enforced in the workflow layer
- ✓ Story quality standards – contract fields enforced by G2
- ✓ Factory activity log – session records in the plane, with durations
- ✓ Traceability matrix – derived_from pointers, walkable both ways
- ✓ Completeness checks (G0-G2) – requirement→story mapping, AC coverage
- ✓ + Accuracy loop – calibration narrows error each epic
- ✓ Four-layer architecture – rules / skills / workflows / orchestrator as directories
- ✓ Provenance ledger – append-only, SHA-256 hash-chained
- ✓ Independent review (G3) – verdict artifacts; labelled rule-based until LLM lands
- ✓ Change management – versioned, ledgered skill amendments
- ✓ Staleness detection – hash mismatch flags downstream; G4 refuses
- ✓ + Self-healing loop – logs trigger skill amendments, next run proves them

MapleSure Insurance is a fictional demo insurer. All data synthetic. Design approved 2026-08-06 – implementation pending; nothing on this page is a claim that the system runs today.